

Caching in on Locality: Spatial and temporal data access in SpMV

Antoine Kempf
Friedrich-Alexander University
Erlangen, Germany
aantoinekempf@icloud.com

Atsushi Suzuki
RIKEN R-CCS
Kobe, Japan
Atsushi.suzuki.aj@a.riken.jp

Masado Alexander Ishii
RIKEN R-CCS
Kobe, Japan
masado.ishii@riken.jp

1 CONTEXT AND MOTIVATION

Sparse Matrix-Vector Multiplication (SpMV) is the computational cornerstone of iterative solvers to find numerical solution of non-linear/linear equation in scientific computing. Especially, partial differential equations for continuous mechanics problems, e.g., fluid flow problem, elasticity problem, heat transfer problem, and coupling of them, are discretized using unstructured mesh for efficient approximation, which leads to nonlinear/linear system with sparse matrix having non-uniform pattern. SpMV reads coefficient of the matrix A and vector data x , performs multiplication and addition operations (fma), and writes obtained value as the result, $y = Ax$. The most common data structure to store the sparse matrix is Compressed Sparse Row (CSR), where only nonzero entries of the sparse matrix are stored with column index and its value. By using FP64 double precision data and 32bit integer for indexing, data accesses are counted as $1.5 \times \text{nnz} + 2.5 \times \text{nrow}$ words in 64bit. Here ‘nnz’ stands for number of nonzero entries and ‘nrow’ for the size of left and right vectors. Since ‘fma’ operation is performed once to all nonzero entries, then the ratio of arithmetic to data access is less than $2/3$. This simple estimate shows SpMV is notoriously memory-bounded operation and it is quite challenging to utilize modern multi-core architectures with SIMD processors. We need to introduce better locality in accessing the main memory and to utilize cache memory efficiently. ‘Communication avoiding’ algorithm[2] was introduced to eliminate idling time of processor cores due to movement of data from other nodes. Typical implementation is seen in s-step GMRES method, where order of orthogonalization processes among search vectors that obtained by multiplication of sparse matrix to the previous vector are rearranged and initiation of inner product using MPI allreduce is aggregated. We now propose a ‘memory access avoiding’ paradigm focusing on memory hierarchy with cache and main memory on the single node. Our goal is to minimize data movement from main memory to cache memory by maximizing both ‘spatial locality’ and ‘temporal locality’ in SpMV operation.

2 IMPROVING LOCALITY

Since sparse matrix has unstructured pattern, blocking technique to improve the locality is based on graph algorithm. Reverse Cuthill-McKee (RCM) ordering is a classical approach and it is introduced to minimize the bandwidth of the matrix and to reduce fill-in entries in factorization procedure. Similar approach to RCM, named ‘level-based blocking’ is introduced[1] in calculation of the Matrix Powers Kernel (MPK). Our method is based on another common technique, nested-dissection (ND) ordering, which is used to extract parallelism as the multi-frontal method for the direct solver of sparse matrix. The global matrix is decomposed into a union of sub-matrices by a standard graph partitioner, METIS[3]. Here sub-matrices in the bottom layer of ND has separated structure with internal and boundary indices, which enforces locality of matrix data structure.

2.1 Spatial Locality

After re-indexing the vector by ND ordering, accessing to vector consists of two phases. One is to perform multiplication of a part of the sub-matrix with internal indices to the vector. Here accessing to the vector is smoothly performed within the same cache line. Here we can enjoy the size effect on smaller matrix which fits inside of the cache memory. The other performs multiplication concerning about boundary indices among sub-matrices. This will not contribute to improving locality, but remains as the original operation without ND ordering.

2.2 Temporal Locality

For MPK operation to generate $[Ax, \dots, A^k x]$ from given vector x , we can expect the coefficient of the matrix data is reused in sequence of the multiplication of the matrix to vectors. Our partitioned sub-matrix by ND ordering allows to apply multiplication of sub-matrix internal indices successively to the next vector before finishing calculation the vector on boundary indices.

2.3 Application-Specific Locality

In finite element approximation to the fluid problem, we exploit coupling structure on the differential operator involving four degrees of freedom per one finite element node, where three velocity components and one pressure are assigned. This results in natural 4×4 dense blocks under the sparse structure and we use a Block-CSR (BSR) format with block size set as 4.

3 PERFORMANCE RESULTS

Multiplication of 4×4 tensor entry of the sparse matrix to 4-length entries is vectorized in column-major order and this size aligns perfectly with 256-bit AVX2 registers for four FP64 data. We have implemented a two-fold strategy to compute $[Ax, A^2 x]$. By numerical experiments, we observe that the ND ordering brings 20% faster calculation than the original code with compiler optimization to use ‘fma’ operation, 4×4 block structure with SIMD vectorization does 45% faster, two-fold strategy does 52% faster. The result without SIMD vectorization is comparable with SpMV routine with inspector-exectuator interface in Intel MKL, and combined three locality achieves faster computation.

REFERENCES

- [1] C. Alappat, G. Hager, O. Schenk, and G. Wellein. 2023. *Level-Based Blocking for Sparse Matrices: Sparse Matrix-Power-Vector Multiplication*. IEEE Transactions on Parallel and Distributed Systems 34, 2 (2023), 581–597. DOI: 10.1109/TPDS.2022.3223512
- [2] J. Demmel, M. Hoemmen, M. Mohiyuddin, and K. Yelick. 2008. *Avoiding communication in sparse matrix computations*. In Proceedings of the 2008 IEEE International Symposium on Parallel and Distributed Processing (IPDPS). IEEE, 1–12.
- [3] G. Karypis and V. Kumar. 1998. *A fast and high quality multilevel scheme for partitioning irregular graphs*. SIAM Journal on Scientific Computing 20, 1 (1998), 359–392.